Document number: P0275R3
Project: Programming Language C++
Audience: Library Evolution Working Group

Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2018-02-01

A Proposal to add Classes and Functions Required for Dynamic Library Load

Changes to P0275R1 and P0275R2 are marked with blue. ☐ Show deleted lines.

> "This is just a means to get a pointer to a function."
>
> — *Ville Voutilainen*

I. Introduction and Motivation

Adding a specific features to an existing software applications at runtime could be useful in many cases. Such extensions, or plugins, are usually implemented using shared library files (.dll, .so/Dynamic Shared Objects) loaded into the memory of program at runtime. Such trechique is called **Dynamic Loading (DL)**.

Current C++ Standard lacks support for Dynamic Library Loading. This proposal attempts to fix that and provide a simple to use classes and functions for DL.

A quick survey at one of the popular Russian IT sites shows that ~41% of people use dynamic loading in company or their own projects. Any wrapper around platform specific DL involves multiple macro, reinterpret_casts, work with platform specific decorations, and even UBs due to OS API designs. Examples: HPX, POCO (see SharedLibrary* files), Qt(see qlibrary_*.cpp files).

This proposal is based on the Boost.DLL library. Unlike many other libraries the Boost.DLL does not force a specific type for the imported symbols, does not include project specific stuff into the classes. It's just a abstraction around platform specific calls that attempts to unify DL behavior.

II. Impact on the Standard

This proposal is a pure library extension. It does not propose changes to existing headers.

This proposal is a library only extension and avoids adding a description of shared libraries machinery as much as possible. EWG has to decide:

- is it acceptable to describe usage of shared library files without any mention of how to construct them?
- is it acceptable to have an unspecified behavior for DLL's TLS, globals, ADL, exceptions, ODR violations, global allocators?

EWG voted on "Add DL support to standard library?" in Albuquerque: 6 | 6 | 4 | 1 | 1, asked to embed feedback and send the proposal to LEWG.

III. Design Decisions

A. Usage of `std::filesystem::path`

To simplify library usage `std::filesystem::path` is used for specifying paths . All the `path` related overhead is minor, comparing to the time of loading shared library file into the memory of program or getting information about shared library file location.

B. Stick to the Filesystem error reporting scheme.

Provide two overloads for some functions, one that throws an exception to report system errors, and another that sets an `error_code`. This supports two common use cases:

- Uses where shared library file related system errors are truly exceptional and indicate a serious failure.
- Uses where shared library file related errors are routine and do not necessarily represent failure.

C. Do not take care of mangling.

C++ symbol mangling depend on compiler and platform. Existing attempts to get mangled symbol by unmangled name result in significant complexity and memory usage growth and overall slowdown of getting symbol from shared library file.

```
[ Example:
    // Loading a shared library file with mangled symbols
    mangled_library lib("libmycpp.so");

    // Attempt to get function "foo::bar" with signature void(int)
    lib.get<void(int)>("foo::bar");

    // The problem is that we do not know what `foo::bar` is:
    // * `foo` could be a class and `bar` could be a static member function
    // * `foo` could be a struct and `bar` could be a static member function
    // * `foo` could be a namespace and `bar` could be function in that namespace
    //
    // We also do not know the calling convention for `foo` and it's noexcept specification.
    //
    // Mangling of `foo::bar` depends on all the knowledge from above, so we are either forced to
    // mangle and try to obtain all the available combinations; or we need to investigate all the
    // symbols exported by "libmycpp.so" shared library file and find a best match.
- end example ]
```

While no good solution was found for obtaining mangled symbol by unmangled name, this proposal concentrates on obtaining symbols by exact name match.

D. Drop the `import` function form Boost.DLL.

While those function could simplify code development for some users, they are simple to missuse:

```
try {
    auto f = dll::import<int()>(path_to_pugin, "function");
    f();
    // `f` goes out of scope along, plugin is unloaded, the exception::what() code is unreachable
} catch (const std::exception& e) {
    std::cerr << e.what();
}
```

E. Do not search libraries in system specific paths by default.

Searching paths for a specified shared library file may affect load times and make the proposed wording less efficient. It is assumed to be the most common case, that user exactly knows were the desired shared library file is located and provides either absolute or relative to current directory path. For that case searching system specific paths affects performance and increases the chance of finding wrong shared library file with the same name. Because some operating systems search system paths even if relative path is provided, the requirement to not do that is implicitly described in proposed wording. That OS specific logic could be avoided by implicitly converting relative paths to absolute.

F. Minimal modifications for the N4687 wording.

There have been DL related proposals:

- N1400: Toward standardization of dynamic libraries
- N1418: Dynamic Libraries in C++
- N1428: Draft Proposal for Dynamic Libraries in C++
- N1496: Draft Proposal for Dynamic Libraries in C++ (Revision 1)
- N1976: Dynamic Shared Objects: Survey and Issues
- N2015: Plugins in C++
- N2407: C++ Dynamic Library Support

The proposal you're reading attempts to avoid issues of the proposals from above, minifying the changes to the N4687 wording. The proposal you're reading:

- is a pure library extension (does not modify Basic Concepts [basic] and does not add new keywords)
- does not add platform requirement for shared libraries support

G. Overloads take `const char*` and `const string&`, not `string_view`.

OS specific API usually accepts symbol name as a zero terminated char array. `string_view` has no zero termination guarantee so it requires copying of the input parameter and zero termination. `char*` and `string&` is a more efficient API design.

IV. Proposed wording relative to N4687

?? Dynamic loading support library[dl]

??.1 General [dl.general]

'Dynamic loading' is a runtime mechanism to load shared library file into the memory of current program at any point of `main`[basic.start.main] execution, retrieve the addresses of functions and objects contained in the shared library file, execute those functions or access those objects, unload the shared library file from memory.

'Shared library file' is a file containing set of objects and functions available for use at program startup or at runtime.

Header <experimental/dl> defines classes and functions suitable for dynamic loading. For that header term 'symbol' relates to a function, reference, class member or object that can be obtained from shared library file at runtime. Term 'symbol name' relates to a character identifier of a symbol, using which symbol can be obtained from shared library file. *[Note:* For symbols declared with `extern "C"` in code of shared library file, 'symbol name' is the name of the entity. 'symbol name' for entities without `extern "C"` are unspecified. Typically symbol names of those entities are mangled entity names — *end note]*.

??.2 Error reporting [dl.errors]

Functions not having an argument of type `error_code&` report errors as follows, unless otherwise specified:

- When a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications, an exception of type system_error is thrown.
- Failure to allocate storage is reported by throwing an exception as described in the C++ standard, 20.5.5.12[res.on.exception.handling].
- Destructors throw nothing.

Functions having an argument of type `error_code&` report errors as follows, unless otherwise specified:

- If a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications, the `error_code&` argument is set as appropriate for the specific error. Otherwise, `clear()` is called on the `error_code&` argument.
- Failure to allocate storage is reported by throwing an exception as described in the C++ standard, 20.5.5.12 [res.on.exception.handling].

??.3 Header <experimental/dl> [dl.dl]

```
namespace std {
  namespace experimental {
  inline namespace dl_v1 {

    // class to work with shared library files
    class shared_library;

    bool operator==(const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator!=(const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator< (const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator> (const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator<=(const shared_library& lhs, const shared_library& rhs) noexcept;
    bool operator>=(const shared_library& lhs, const shared_library& rhs) noexcept;

    // free functions
    template<class T>
    filesystem::path symbol_location(const T& symbol, error_code& ec);
    template<class T>
    filesystem::path symbol_location(const T& symbol);

    filesystem::path this_line_location(error_code& ec);
    filesystem::path this_line_location();

    filesystem::path program_location(error_code& ec);
    filesystem::path program_location();

  }
  }

  // support
  template <class T> struct hash;
  template <> struct hash<experimental::shared_library>;
}
```

??.4 Class shared_library [dl.shared_library]

The class `shared_library` provides means to load shared library file into the memory of program, check that shared library file exports symbol with specified symbol name and obtain that symbol.

`shared_library` instances share reference count to an actual shared library file loaded in memory of current program, so it is safe and memory efficient to have multiple instances of `shared_library` referencing the same shared library file even if those instances were loaded in memory using different paths (relative and absolute) referencing the same shared library file.

[Note: It is safe to concurrently load shared library files into memory of program, unload and get symbols from any shared library files using different `shared_library` instances. If current platform does not guarantee safe concurrent work with shared library files, calls to `shared_library` functions are serialized by `shared_library` implementation. — *end note]*

[Note: Constructors, comparisons and `reset()` functions that accept `nullptr_t` are not provided because that may cause confusion: some of the platforms provide interfaces for shared library files that accept `nullptr_t` to get a handler to the current program. — *end note]*

```
namespace std {
  namespace experimental {
  inline namespace dl_v1 {

    class shared_library {
    public:
      using native_handle_type = platform-specific;

      // shared library file load modes
      using dl_mode = T1;
      static constexpr dl_mode default_mode;
      static constexpr dl_mode rtld_lazy;
      static constexpr dl_mode rtld_now;
      static constexpr dl_mode rtld_global;
      static constexpr dl_mode rtld_local;
      static constexpr dl_mode add_decorations;
      static constexpr dl_mode search_system_directories;

      // construct/copy/destruct
      shared_library() noexcept;
      shared_library(shared_library&& lib) noexcept;

      explicit shared_library(const filesystem::path& library_path);
      shared_library(const filesystem::path& library_path, dl_mode mode);
      shared_library(const filesystem::path& library_path, error_code& ec);
      shared_library(const filesystem::path& library_path, dl_mode mode, error_code& ec);
      ~shared_library();

      // public member functions
      shared_library& operator=(shared_library&& lib) noexcept;
      void reset() noexcept;
      explicit operator bool() const noexcept;

      template <typename SymbolT>
      SymbolT* get_if(const char* symbol_name) const noexcept;

      template <typename SymbolT>
      SymbolT* get_if(const string& symbol_name) const noexcept;


      template <typename SymbolT>
      SymbolT& get(const char* symbol_name) const;

      template <typename SymbolT>
      SymbolT& get(const string& symbol_name) const;

      native_handle_type native_handle() const noexcept;

      // public static member functions
      static constexpr bool platform_supports_dl() noexcept;
      static constexpr bool platform_supports_dl_of_program() noexcept;
    };

  }
  }
}
```

`shared_library` defines member bitmask type `dl_mode`.

??.4.1 Type dl_mode [dl.shared_library.dl_mode]

```
using dl_mode = T1;
```

Bitmask type `dl_mode` provides modes for searching the shared library file and platform specific modes for loading the shared library file in memory of program. *[Note:* library users may extend available modes by casting the required platform specific mode to dl_mode: *[Example:* `static_cast<dl_mode>(RTLD_NODELETE);` — *end example]* — *end note]*

??.4.2 dl_mode constants [dl.shared_library.dl_const]

`add_decorations`

Effects: Adds a platform specific extensions and prefixes to shared library filename before trying to load it into program memory. If load attempts fail, loads with exactly specified name.

Value: Any value that can not be received by applying binary OR to any set of modes from `dl_mode`

[Example:

```
// Attempts to open
//      `./my_plugins/plugin1.dl` and `./my_plugins/libplugin1.dl` on Windows
//      `./my_plugins/libplugin1.so` on Linux
//      `./my_plugins/libplugin1.dylib` and `./my_plugins/libplugin1.so` on MacOS.
// If that fails, loads `./my_plugins/plugin1`
shared_library lib("./my_plugins/plugin1", dl_mode::add_decorations);
```

- end example]

`rtld_lazy`

Effects: As if applying RTLD_LAZY on POSIX.

Value: `0` if platform does not have RTLD_LAZY equivalent. Platform specific value of RTLD_LAZY equivalent otherwise.

`rtld_now`

Effects: As if applying RTLD_NOW on POSIX.

Value: `0` if platform does not have RTLD_NOW equivalent. Platform specific value of RTLD_NOW equivalent otherwise.

`rtld_global`

Effects: As if applying RTLD_GLOBAL on POSIX.

Value: `0` if platform does not have RTLD_GLOBAL equivalent. Platform specific value of RTLD_GLOBAL equivalent otherwise.

`rtld_local`

Effects: As if applying RTLD_LOCAL on POSIX.

Value: `0` if platform does not have RTLD_LOCAL equivalent. Platform specific value of RTLD_LOCAL equivalent otherwise.

`default_mode`

Value: `rtld_lazy | rtld_local`.

`search_system_directories`

Value: Any value that can not be received by applying binary OR to any set of modes from `dl_mode`

Effects: Allows loading of shared library files from system specific shared library file directories [fs.def.directory] along with loading shared library files from current directory.

??.4.3 shared_library constructors [dl.shared_library.constr]

```
shared_library() noexcept;
```

Effects: Creates `shared_library` that does not reference any shared library file.

Postconditions: `*this` is `false`.

```
shared_library(shared_library&& lib) noexcept;
```

Effects: Assigns the state of `lib` to `*this` and sets `lib` to a default constructed state.

Remarks: Does not invalidate symbols previously obtained from `lib`.

Postconditions: `lib` is `false`, `*this` is `true`.

```
explicit shared_library(const filesystem::path& library_path);
shared_library(const filesystem::path& library_path, dl_mode mode);
shared_library(const filesystem::path& library_path, error_code& ec);
shared_library(const filesystem::path& library_path, dl_mode mode, error_code& ec);
```

Effects: If the shared library file specified by `library_path` is already loaded in memory of current program makes `*this` reference the previously loaded shared library file. Otherwise loads a shared library file by specified path with a specified mode into the memory of current program, executes all the platform specific initializations.

References current program if absolute or relative path to the program was provided as `library_path` and `platform_supports_dl_of_program()` is `true`.

Loads shared library file with changed name and applied extension only if `(mode & dl_mode::add_decorations)` is not 0. Loads a shared library file from system specific shared library file directories only if `library_path` contains only shared library filename and `(mode & dl_mode::search_system_directories)` is not 0. During loads from system specific directories file name modification rules from above apply. If with `dl_mode::search_system_directories` more than one shared library file has the same base name and extension, the function loads in memory any first matching shared library file. If `(mode & dl_mode::search_system_directories)` is 0, then any relative path or path that contains only filename is treated as a path in current working directory.

Library open mode is equal to `(mode & ~dl_mode::search_system_directories & ~dl_mode::add_decorations)` converted to a platform specific type representing shared library file load modes and adjusted to satisfy the `dl_mode::search_system_directories` requirements from above. If mode is invalid for current platform, attempts to adjust the mode by applying `dl_mode::rtld_lazy` and reports error if resulting mode is invalid. *[Example:* If mode on POSIX was set to `rtld_local`, then it will be adjusted to `rtld_lazy | rtld_local`. - *end example]*

Throws: As specified in [dl.errors]

??.4.4 shared_library destructor [dl.shared_library.destr]

```
~shared_library();
```

Effects: Destroys the `shared_library` by calling `reset()`.

```
shared_library& operator=(shared_library&& lib) noexcept;
```

Effects: If `*this` then calls `reset()`. Assigns the state of `lib` to `*this` and sets `lib` to a default constructed state.

Remarks: Does not invalidate symbols previously obtained from `lib`.

Postconditions: `lib` is `false`, `*this` is `true`.

Returns: `*this`

??.4.6 shared_library members[dl.shared_library.member]

```
void reset() noexcept;
```

Effects: Sets `*this` to a default constructed state. If `*this` was the last instance of `shared_library` referencing the shared library file, then all the resources associated with shared library file shall be released.

Remarks: Symbols obtained from shared library file remain valid if there is at least one instance of `shared_library` referencing the shared library file.

Postconditions: `*this` is `false`.

```
explicit operator bool() const noexcept;
```

Returns: `true` if `*this` references a shared library file.

```
template <typename SymbolT>
  SymbolT* get_if(const char* symbol_name) const noexcept;

template <typename SymbolT>
  SymbolT* get_if(const string& symbol_name) const noexcept;
```

Returns: Pointer to symbol from shared library file that has the name `symbol_name`, `nullptr` otherwise.

Remarks: It's the user responsibility to provide valid `SymbolT` type for symbol. However implementations may provide additional checks for matching `SymbolT` type and actual symbol type. *[Note:* For example implementations may check that `SymbolT` is a function pointer and that symbol with `symbol_name` allows execution. — *end note]*

```
template <typename SymbolT>
  SymbolT& get(const char* symbol_name) const;

template <typename SymbolT>
  SymbolT& get(const string& symbol_name) const;
```

Returns: `*get_if<SymbolT>(symbol_name)`.

Throws: `system_error` if symbol does not exist or if the shared library file was not loaded.

```
native_handle_type native_handle() const noexcept;
```

Returns: Native handler of the loaded in memory shared library file or default constructed native_handle_type if *this does not reference a shared library file. *[Note:* This member allow implementations to provide access to implementation details. Actual use of these members is inherently non-portable. — *end note]*

??.4.7 shared_library static members[dl.shared_library.static]

```
static constexpr bool platform_supports_dl() noexcept;
```

Returns: true if platform supports loading of shared library files into the memory of program. [Note: If this function returns `false`, then any attempt to load a shared library file will fail at runtime. This function could be used to ensure platform capabilities at compile time: `static_assert(shared_library::platform_supports_dl(), "DL required for this program");` — end note]

```
static constexpr bool platform_supports_dl_of_program() noexcept
```

Returns: `true` if according to platform capabilities `shared_library(program_location())` may succeed.

??.4.8 shared_library free operators[dl.shared_library.operators]

`shared_library` provides fast comparison operators that compare the referenced shared libraries file.

```
bool operator==(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: `lhs.native_handle() == rhs.native_handle()`

```
bool operator!=(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: `lhs.native_handle() != rhs.native_handle()`

```
bool operator<(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: `lhs.native_handle() < rhs.native_handle()`

```
bool operator>(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: `lhs.native_handle() > rhs.native_handle()`

```
bool operator<=(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: `lhs.native_handle() <= rhs.native_handle()`

```
bool operator>=(const shared_library& lhs, const shared_library& rhs) noexcept;
```

Returns: `lhs.native_handle() >= rhs.native_handle()`

??.4.9 shared_library hash support[dl.shared_library.hash]

```
template <> struct hash<experimental::shared_library>;
```

The specialization is enabled (23.14.15).

??.5 Runtime path functions[dl.location]

```
template<class T>
filesystem::path symbol_location(const T& symbol, error_code& ec);
template<class T>
filesystem::path symbol_location(const T& symbol);
```

Returns: Full path and name to shared library file or program that contains `symbol`

Throws: As specified in [dl.errors]

```
filesystem::path this_line_location(error_code& ec);
filesystem::path this_line_location();
```

Returns: Full path and name to shared library file or program that contains line of code in which `this_line_location()` was called

Throws: As specified in [dl.errors]

```
filesystem::path program_location(error_code& ec);
filesystem::path program_location();
```

Returns: Full path and name to the current program.

Throws: As specified in [dl.errors]

V. Feature-testing macro

For the purposes of SG10, we recommend the feature-testing macro name __cpp_lib_shared_library.

VI. Revision History

Revision 3:

- Changed Dynamic Library Load(DLL) term to Dynamic Loading (DL)
- Embedded EWG voting results
- Dropped copy constructors, assignment operators and assign functions of shared_library
- Dropped shared_library::location functions

Revision 2:

- Added EWG questions
- Added motivation for not using `string_view`
- Return pointer from get
- Some usage numbers and examples
- Specified and reduced load modes
- Fixed "must" usage in proposal
- Dropped `load()` functions
- Revised constructor overloads
- Dropped `import` functions
- Added feature testing macro
- "append_decorations" => "add_decorations"
- `dl_mode` description rewritten
- Multiple minor improvements
- Rebased to N4687

Revision 1:

- Rebased to N4618 and applied minor fixes

Revision 0:

- Initial proposal

VII. References

[Boost.DLL] Boost DLL library. Available online at https://github.com/boostorg/dll

[N4618] Working Draft, Standard for Programming Language C++. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4618.pdf

[N1400] Toward standardization of dynamic libraries. Available online at http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2002/n1400.html

[N1418] Dynamic Libraries in C++. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2002/n1418.html

[N1428] Draft Proposal for Dynamic Libraries in C++. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1428.html

[N1496] Draft Proposal for Dynamic Libraries in C++ (Revision 1). Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2003/n1496.html

[N1976] Dynamic Shared Objects: Survey and Issues. Available online at http://www.open-

std.org/jtc1/sc22/wg21/docs/papers/2006/n1976.html

[N2015] Plugins in C++. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2006/n2015.pdf

[N2407] C++ Dynamic Library Support. Available online at http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2407.html

VIII. Acknowledgements

Klemens Morgenstern highlighted some of the missing functionality in Boost.DLL and provided implementation of mangled symbols load, that showed complexities of such approach. Renato Tegon Forti started the work on Boost.DLL and provided a lot of code, help and documentation for the Boost.DLL.

Thanks to Tom Honermann for providing comments and recommendations.